

20. Видове графи и представяне на графи.

Алгоритми за търсене в дълбочина и широчина.

Ойлерови и Хамилтонови цикли в граф.

1. Увод

Теорията на графите е дял от дискретната математика, който изучава структури от обекти (върхове) и връзките между тях (ребра). Граф моделите са фундаментални за компютърните науки - те се използват за представяне на компютърни мрежи, социални мрежи, транспортни системи, зависимости между задачи (планиране), карти на пътища

2. Основни понятия и видове графи

2.1 Формално определение

Граф се дефинира като наредена двойка $G = (V, E)$, където:

- V е крайно множество от върхове (vertices или nodes)
- E е крайно множество от ребра (edges), като всяко ребро свързва двойка върхове

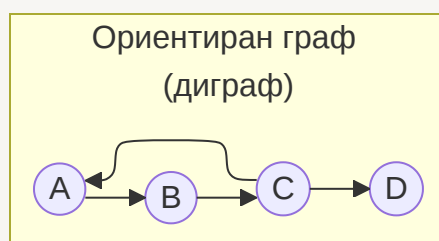
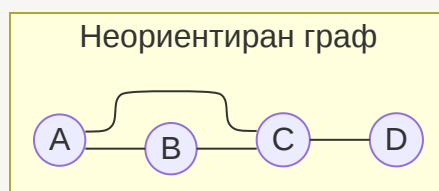
Броят на върховете обикновено се бележи с $n = |V|$, а броят на ребрата се бележи с $m = |E|$.

$$V = \{v_1, v_2, \dots, v_n\}$$

$$E = \{e_1, e_2, \dots, e_m\}$$

2.2 Ориентирани и неориентирани графи

- При **неориентирани графи** ребрата са неподредени двойки $\{u, v\}$ - връзката е симетрична (ако u е свързан с v , то v е свързан с u)
- При **ориентирани графи** (диграф) ребрата (наричани дъги) са наредени двойки (u, v) , представляващи връзка, насочена от u към v . $(u, v) \neq (v, u)$



2.3 Претеглени и непретеглени графи

Ако на всяко ребро е съпоставено число (тегло), чрез функция $w : E \rightarrow \mathbb{R}$ графът е **претеглен**. Теглата обикновено моделират разстояние, цена, капацитет или време. В противен случай графът е **непретеглен**.

2.4 Прости графи, мултиграфи и псевдографи

- **Прост граф** - без примки (ребро от връх към самия себе си) и без повече от едно ребро между една и съща двойка върхове
- **Мултиграф** - допуска се съществуването на повече от едно ребро между една и съща двойка върхове
- **Псевдограф** - Мултиграф, който позволява наличието на примки - ребра, които свързват даден връх със самия него ($e = (u, u)$)

2.5 Степен на връх

Инцидентност: Реброто $e = (u, v)$ е инцидентно на върховете u и v .

За неориентиран граф степента $\deg(v)$ на връх v е броят на инцидентните му ребра.

$$\sum \deg(v) = 2 | E |$$

тъй като всяко ребро допринася по 1 към степента на всеки от двата си края.

За ориентиран граф разграничаваме **входяща степен** $\deg^-(v)$ (брой дъги, влизащи във v) и **изходяща степен** $\deg^+(v)$ (брой дъги, излизащи от v), като:

$$\sum_{v \in V} \deg^+(v) = \sum_{v \in V} \deg^-(v) = |E|$$

2.6 Пълен и двуделен граф

- **Пълен граф** - всеки два различни върха са свързани с ребро.
Броят на ребрата е $\binom{n}{2} = \frac{n(n-1)}{2}$.
- **Двуделен граф** - граф, чието множество от върхове V може да се раздели на две непресичащи се подмножества V_1 и V_2 ($V = V_1 \cup V_2$, $V_1 \cap V_2 = \emptyset$), така че всяко ребро $e \in E$ има единия си край в V_1 , а другия - в V_2 .

2.7 Дървета и ациклични графи

Дърво е неориентиран граф без цикли. Дърво с n върха има точно $n - 1$ ребра. Ориентиран граф без насочени цикли се нарича **DAG (Directed Acyclic Graph)**.

3. Представяне на графи

3.1 Матрица на съседство

Квадратната матрица A с размер $n \times n$, където:

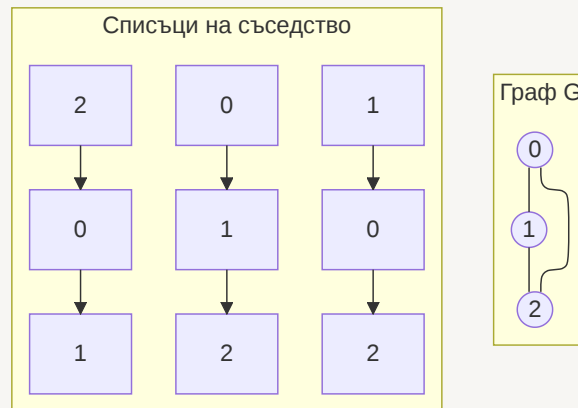
$$A[i][j] = \begin{cases} 1 \text{ (или } w(i, j)), & \text{ако съществува ребро } (i, j), \\ 0 \text{ (или } \infty), & \text{иначе.} \end{cases}$$

- Заема памет $O(|V|^2)$ независимо от броя на ребрата
- Проверка за съществуване на конкретно ребро - $O(1)$
- Изброяване на всички съседни върхове - $O(|V|)$

3.2 Списъци на съседство

Това представяне се състои от масив Adj от $|V|$ на брой линейни списъка. Всеки списък $Adj[u]$ съдържа всички върхове v , за които съществува ребро $(u, v) \in E$.

- Заема памет $O(|V| + |E|)$ - пропорционална на действителния брой ребра
- Изброяване на съседите на връх v - $O(deg(v))$
- Проверка за съществуване на конкретно ребро - $O(deg(v))$ в най-лошия случай



3.3 Матрица на инцидентност

Двумерен масив B с размери $|V| \times |E|$. За неориентиран граф:

$$B[i][j] = \begin{cases} 1, & \text{ако връх } v_i \text{ е инцидентен на ребро } e_j \\ 0, & \text{в противен случай} \end{cases}$$

За ориентирани графи често се използва $-1 / +1$ за начало/край на дъгата.

Заема $O(|V| \cdot |E|)$ памет и рядко се използва на практика, но е полезна в теоретични доказателства.

3.4 Сравнение

Операция	Матрица на съседство	Списъци на съседство
Памет	$O(V ^2)$	$O(V + E)$
Проверка за ребро (u, v)	$O(1)$	$O(\deg(u))$
Изброяване на съседни	$O(V)$	$O(\deg(u))$
Добавяне на ребро	$O(1)$	$O(1)$
Препоръчителна употреба	гъсти графи	разредени графи

4. Търсене в дълбочина (Depth-First Search, DFS)

4.1 Идея

DFS се стреми да навлезе колкото се може "по-дълбоко" в графа, изследвайки ребрата, излизащи от последно открития връх преди да се върне обратно (backtracking). Алгоритъмът по своята същност е рекурсивен и използва структурата **Стек (Stack)**.

4.2 Реализация

```
DFS(G)
    за всеки връх u от G.V:
        u.color = WHITE
        u.pi = NIL
    time = 0
    за всеки връх u от G.V:
        ако u.color == WHITE:
            DFS-VISIT(G, u)

DFS-VISIT(G, u)
    time = time + 1
    u.d = time
    u.color = GRAY           // Върхът става СИБ (открит)

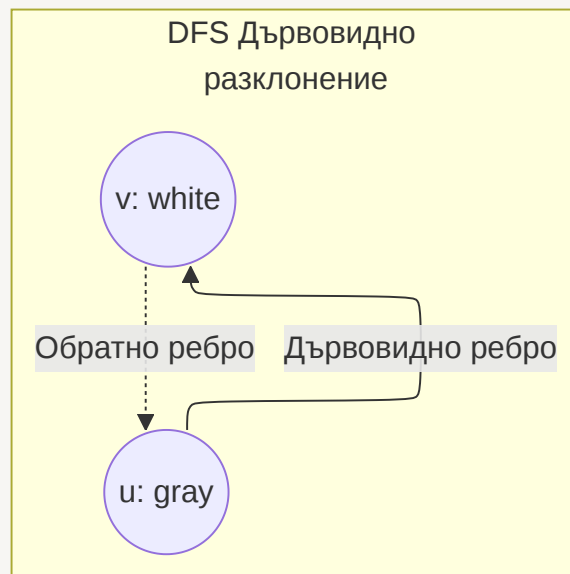
    за всеки v от G.Adj[u]:
        ако v.color == WHITE:
            v.pi = u
            DFS-VISIT(G, v)
    още ако v.color == GRAY:
        // ОТКРИТО ОБРАТНО РЕБРО! Графът има цикъл!
```

```
u.color = BLACK      // Върхът става ЧЕРЕН (завършен)
time = time + 1
u.f = time
```

4.3 Класификация на ребрата

В ориентиран граф всяко ребро, срещнато по време на DFS, се класифицира в една от четири категории:

- **Ребро на дървото (tree edge)** - води до непосетен (бял) връх
- **Обратно ребро (back edge)** - води до предшественик в текущото дърво (сив връх) - наличието му е признак за цикъл
- **Право ребро (forward edge)** - води до вече обходен съсед
- **Кръстосано ребро (crossed edge)** - свързва върхове, между които няма отношение



В **неориентиран** граф не съществуват прави и кръстосани ребра - при него се срещат единствено tree edges и back edges, тъй като всяко ребро се разглежда от двете посоки.

Теорема за скобите (Parenthesis Theorem): За всеки два върха u и v , интервалите $[d[u], f[u]]$ и $[d[v], f[v]]$ са или напълно вложени един в друг, или напълно разделени.

4.4 Сложност

При представяне със списъци на съседство $O(|V| + |E|)$, защото всеки връх се посещава веднъж и всяко ребро се разглежда точно веднъж. При матрицата на съседство сложността е $O(|V|^2)$.

5. Търсене в широчина (Bread-First Search, BFS)

5.1 Идея

BFS е стратегия за обхождане на нива. Алгоритъмът започва от зададен връх-източник s и посещава всички негови директни съседни, след това техните съседни и т.н. Използва се структурата **Опашка (Queue)** на принципа FIFO (First-In-First-Out).

За управление на състоянията се въвеждат три цвята:

- **БЯЛ (WHITE):** Непосетен връх.
- **СИВ (GRAY):** Посетен връх, чиито съседни все още не са изследвани напълно (в опашката е).
- **ЧЕРЕН (BLACK):** Напълно изследван връх (изваден от опашката).

5.2 Псевдокод

```
BFS( $G, s$ )
  за всеки връх  $u$  от  $G.V - \{s\}$ :
     $u.color = WHITE$ 
     $u.d = INF$ 
     $u.pi = NIL$ 
```

```

s.color = GRAY
s.d = 0
s.pi = NIL

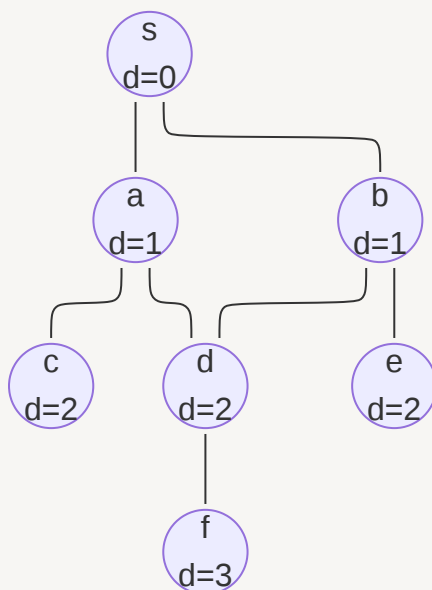
Q = Нова_Опашка()
ENQUEUE(Q, s)

докато Q не е празна:
    u = DEQUEUE(Q)
    за всеки v от G.Adj[u]:
        ако v.color == WHITE:
            v.color = GRAY
            v.d = u.d + 1
            v.pi = u
            ENQUEUE(Q, v)
    u.color = BLACK

```

5.3 Свойства и намиране на най-къси пътища

Ключово свойство: в **непретеглен** граф BFS от начален връх s намира дължината на най-късия път (в брой ребра) от s до всеки достижим връх.



5.4 Сложност

Аналогично на DFS: $O(|V| + |E|)$ със списъци или $O(V^2)$ с матрица на съседство - всеки връх се поставя в опашката точно веднъж, а всяко ребро се разглежда константен брой пъти.

6. Ойлерови цикли

6.1 Определения

- **Ойлеров път** - път в граф, който преминава през **всяко ребро** точно веднъж
- **Ойлеров цикъл** - Ойлеров път, който започва и завършва в един и същ връх

6.2 Теорема на Ойлер

За свързан неориентиран граф G :

1. Графът притежава **Ойлеров цикъл** тогава и само тогава, когато степента на всеки връх е четно число ($\forall v \in V, \deg(v) \equiv 0 \pmod{2}$).

2. Графът притежава **Ойлеров път** тогава и само тогава, когато има точно **два върха** с нечетна степен. Пътят задължително започва в единия нечетен връх и завършва в другия.

За ориентиран граф, изискването е графът да бъде силно свързан (ако игнорираме изолираните върхове) и за всеки връх да е изпълнено:

$\deg^+(v) = \deg^-(v)$ за **цикъл**, или за точно два върха да има отместване с ± 1 за **път**.

6.3 Алгоритмично намиране

Ако критерият на Ойлер е изпълнен, цикълът може да се намери за линейно време $O(|E|)$ чрез алгоритъма:

1. Избира се произволен начален връх v .
2. Изгражда се подцикъл, движейки се по неизползвани ребра, докато не се върнем във v (засядане). Тъй като степените са четни, засядането винаги е в началния връх.
3. Ако все още има неизползвани ребра, избира се връх от текущия цикъл, който има инцидентни неизползвани ребра, и се повтаря процедурата, след което новата траектория се "вплита" в основната.

7. Хамилтонови пътища и цикли

7.1 Определения

- **Хамилтонов път** - път, който посещава **всеки връх** в графа точно веднъж
- **Хамилтонов цикъл** - Хамилтонов път, който се затваря обратно към началния връх

7.2 Разлика спрямо Ойлеровите цикли

	Ойлеров цикъл	Хамилтонов цикъл
Условие	използва всяко ребро веднъж	посещава всеки върх веднъж
Характеризация	пълна (теорема на Ойлер)	няма известна проста необходима и достатъчна характеристика
Сложност на разпознаването	$O(V+E)$ (полиномиална)	NP-пълна задача

7.3 Достатъчни условия

Тъй като няма проста характеристика, са изведени само **достатъчни** (но не и необходими) условия за съществуване на Хамилтонов цикъл:

1. Ако G е прост граф с $n \geq 3$ върха и за всеки връх $v \in V$ е изпълнено

$$\deg(v) \geq \frac{n}{2}$$

2. Ако G е прост граф с $n \geq 3$ върха и за всяка двойка несвързани (несъседни) върхове u и v е в сила: $\deg(u) + \deg(v) \geq n$